

MODELOS Y SERVICIOS DE DATOS [MYSD-MBDA]

PL/ SQL Básico

2026-1

Guía autoestudio 4/6

OBJETIVO

1. Conocer herramientas que facilitan el trabajo del desarrollador de una base de datos específicamente la herramienta SQL Developer
2. Desarrollar competencias para definir e implementar restricciones de integridad con mecanismos declarativos y procedimentales.

TÓPICOS OBJETIVO 2

1. Acciones referenciales
2. Disparadores
3. Constantes y variables
4. Instrucciones básicas: asignación
5. Cursores: implícitos y explícitos

ENTREGA

Publicar las respuestas en el espacio correspondiente en un archivo **.zip** , el nombre de este archivo debe ser la concatenación en orden alfabético de los primeros apellidos de cada uno de los miembros.

INVESTIGACIÓN

A. Acciones referenciales

1. ¿Para qué sirven las acciones referenciales?
2. ¿Qué acciones soporta ORACLE? ¿Qué permite hacer cada una de ellas?

B. PL/SQL

1. ¿Qué es PL/SQL?
2. ¿Qué motores lo soportan?

C. Datos e instrucciones en PL/SQL

1. ¿Cuáles son los tipos de datos que ofrece?
2. ¿Cuál es la forma de definir constantes y variables?
3. ¿Cómo se define una variable con un tipo tomado de la base de datos?
4. ¿Cuál es la forma de los diferentes tipos de asignación? (Son tres)

D. Cursores

1. ¿Qué es un cursor implícito? ¿Para qué sirve?
2. ¿Qué es un cursor explícito? ¿Para qué sirve?
3. ¿Cuáles son las excepciones propias de uso de estos cursores?

E. Modularidad

1. ¿Cuál es la estructura general de un bloque PL/SQL?
2. ¿Para qué sirven las diferentes estructuras modulares? (bloque anónimo, procedimiento, función y disparador)

A. Acciones Referenciales

A.1 ¿Para qué sirven las acciones referenciales?

Las acciones referenciales (también llamadas acciones de integridad referencial) son mecanismos que el motor de base de datos ejecuta automáticamente cuando se realiza una operación DML (INSERT, UPDATE, DELETE) sobre una tabla padre que tiene filas relacionadas en una tabla hija mediante una restricción FOREIGN KEY. Su propósito es mantener la consistencia e integridad de los datos: garantizar que no existan filas huérfanas en la tabla hija apuntando a valores que ya no existen en la tabla padre.

Sin acciones referenciales, el desarrollador tendría que gestionar manualmente la propagación de cambios en toda la jerarquía de tablas, lo cual es propenso a errores. Con ellas, el motor de base de datos asume esa responsabilidad de manera automática y transparente.

A.2 ¿Qué acciones soporta ORACLE y qué permite hacer cada una?

Oracle Database soporta las siguientes acciones referenciales, definidas en la cláusula ON DELETE (y con restricciones en ON UPDATE):

Acción	Descripción	Ejemplo de uso
NO ACTION / RESTRICT (por defecto)	Si existen filas hijas relacionadas, la operación sobre la fila padre es rechazada. Oracle lanza un error ORA-02292.	Impedir que se borre un tipo de habitación que tiene habitaciones asignadas.
ON DELETE CASCADE	Al eliminar una fila padre, todas las filas hijas relacionadas se eliminan automáticamente.	Al borrar un Room_type, se eliminan en cascada todas las habitaciones (Room) de ese tipo.
ON DELETE SET NULL	Al eliminar una fila padre, la columna FK en las filas hijas se actualiza a NULL (la columna debe admitir nulos).	Al eliminar un empleado supervisor, poner en NULL el campo supervisor_id de sus subordinados.
ON DELETE SET DEFAULT	Al eliminar una fila padre, la columna FK en las filas hijas toma el valor DEFAULT definido para esa columna. (Soportado desde Oracle 12c).	Al borrar una categoría, asignar la categoría 'General' como categoría por defecto.

Nota importante: Oracle NO soporta ON UPDATE CASCADE de forma declarativa. Para propagar cambios en actualizaciones se deben usar disparadores (triggers).

B. PL/SQL

B.1 ¿Qué es PL/SQL?

PL/SQL (Procedural Language / Structured Query Language) es la extensión procedimental propietaria de Oracle para SQL. Combina la capacidad de consulta y manipulación de datos de SQL con las estructuras de control propias de un lenguaje de programación (condicionales, ciclos, manejo de excepciones, modularidad), todo dentro del motor de la base de datos.

Sus principales características son:

- Ejecución del lado del servidor: el código se compila y ejecuta dentro del motor Oracle, reduciendo el tráfico de red.
- Integración nativa con SQL: las sentencias DML y SELECT se escriben directamente en el bloque PL/SQL sin adaptadores adicionales.
- Manejo estructurado de excepciones: permite capturar y gestionar errores de forma elegante.
- Modularidad: soporta procedimientos almacenados, funciones, paquetes y disparadores (triggers).
- Portabilidad dentro del ecosistema Oracle: cualquier base de datos Oracle ejecuta el mismo código PL/SQL.

B.2 ¿Qué motores soportan PL/SQL?

PL/SQL es exclusivo del ecosistema Oracle. Los motores que lo soportan son:

Motor / Producto	Descripción
Oracle Database (todas las ediciones)	Soporte completo: SE, EE, XE, RAC, etc.
Oracle Database In-Memory	Misma sintaxis, con optimizaciones columnar.
Oracle Autonomous Database	Servicio gestionado en Oracle Cloud, soporta PL/SQL íntegramente.
Oracle TimesTen In-Memory Database	Base de datos in-memory de Oracle, compatible con PL/SQL.
Oracle Berkeley DB (con SQL)	Soporte limitado de PL/SQL vía capa de compatibilidad.

Nota: PostgreSQL tiene PL/pgSQL (similar en sintaxis pero diferente). MySQL tiene un lenguaje procedimental propio. Ninguno de estos es PL/SQL de Oracle.

C. Datos e Instrucciones en PL/SQL

C.1 ¿Cuáles son los tipos de datos que ofrece?

PL/SQL ofrece una amplia variedad de tipos de datos, agrupados en las siguientes categorías:

Escalares

Categoría	Tipos principales	Notas
Numéricos	NUMBER(p,s), INTEGER, FLOAT, BINARY_INTEGER, PLS_INTEGER, SIMPLE_INTEGER	NUMBER es el más usado para valores exactos.
Carácter	VARCHAR2(n), CHAR(n), NVARCHAR2(n), NCHAR(n)	VARCHAR2 es el estándar para texto variable.
Fecha/Hora	DATE, TIMESTAMP, INTERVAL YEAR TO MONTH, INTERVAL DAY TO SECOND	DATE incluye hora en Oracle.
Booleano	BOOLEAN	Solo en PL/SQL; no existe en SQL de Oracle.
LOB	CLOB, BLOB, NCLOB, BFILE	Para objetos grandes (texto, binario).
RAW	RAW(n), LONG RAW	Para datos binarios en bruto.

Compuestos

- RECORD: agrupa campos de distintos tipos en una sola variable (equivalente a una estructura/struct).
- TABLE (colección index-by / associative array): arreglo indexado por INTEGER o VARCHAR2.
- %ROWTYPE: tipo compuesto derivado de una fila completa de tabla o cursor.

Referencias

- REF CURSOR: tipo cursor de referencia, permite retornar cursores como parámetros.

C.2 ¿Cuál es la forma de definir constantes y variables?

Las variables y constantes se declaran en el bloque DECLARE del programa PL/SQL.

Sintaxis general de variable:

```
nombre_variable    tipo_de_dato [NOT NULL] [:= valor_inicial |  
DEFAULT valor_inicial];
```

Sintaxis general de constante:

```
nombre_constante  CONSTANT tipo_de_dato := valor;
```

Ejemplos:

```
DECLARE  
    -- Variable simple  
    v_nombre          VARCHAR2(100);  
    v_sueldo          NUMBER(10,2) := 0;  
  
    -- Variable NOT NULL (requiere valor inicial)  
    v_activo           BOOLEAN NOT NULL := TRUE;  
  
    -- Constante  
    c_iva              CONSTANT NUMBER(5,2) := 19.00;  
    c_pi               CONSTANT NUMBER := 3.14159265;
```

Las variables pueden llevar un valor DEFAULT o asignarse con :=. Las constantes SIEMPRE deben inicializarse en la declaración y no pueden cambiar su valor en tiempo de ejecución.

C.3 ¿Cómo se define una variable con un tipo tomado de la base de datos?

PL/SQL ofrece dos atributos especiales que permiten heredar tipos directamente desde la base de datos, lo que hace el código más robusto ante cambios de esquema:

%TYPE – hereda el tipo de una columna o variable

Permite declarar una variable con exactamente el mismo tipo de dato que tiene una columna específica de una tabla, o que otra variable ya declarada.

```
DECLARE  
    v_room_id          Room.id%TYPE;           -- mismo tipo que la columna  
id de Room  
    v_max_occ          Room.max_occupancy%TYPE;  
    v_tipo             Room_type.id%TYPE;  
BEGIN  
    SELECT id, max_occupancy INTO v_room_id, v_max_occ  
    FROM Room WHERE id = 101;  
END;
```

%ROWTYPE – hereda la estructura completa de una fila

Permite declarar una variable con la estructura completa de una tabla o cursor (todos sus campos con sus tipos respectivos).

```
DECLARE
    v_habitacion    Room%ROWTYPE;           -- estructura completa de
una fila de Room
BEGIN
    SELECT * INTO v_habitacion FROM Room WHERE id = 101;
    DBMS_OUTPUT.PUT_LINE(v_habitacion.max_occupancy);
END;
```

La ventaja es que si el DBA modifica el tipo de una columna, el código PL/SQL que usa %TYPE o %ROWTYPE se adapta automáticamente al recompilarse.

C.4 ¿Cuál es la forma de los diferentes tipos de asignación? (Son tres)

En PL/SQL existen tres formas de asignar un valor a una variable:

Forma 1 – Asignación directa con el operador :=

Se usa para asignar un valor literal, una expresión aritmética o el resultado de una función a una variable.

```
DECLARE
    v_precio    NUMBER := 150000;
    v_total     NUMBER;
BEGIN
    v_total := v_precio * 1.19;           -- expresión aritmética
    v_total := ROUND(v_total, 2);        -- resultado de función
END;
```

Forma 2 – Asignación mediante SELECT INTO

Se usa para asignar el resultado de una consulta SQL a una o varias variables. La consulta debe retornar exactamente una fila; si retorna 0 o más de una fila, se lanza una excepción.

```
DECLARE
    v_max    Room.max_occupancy%TYPE;
    v_tipo   Room.room_type_id%TYPE;
BEGIN
    SELECT max_occupancy, room_type_id
        INTO v_max, v_tipo
        FROM Room
        WHERE id = 205;
END;
```

Forma 3 – Asignación mediante FETCH INTO (con cursores explícitos)

Se usa junto con un cursor explícito. Primero se abre el cursor con OPEN, luego se usa FETCH INTO para traer fila por fila y asignarla a variables o a un registro %ROWTYPE.

```
DECLARE
    CURSOR c_rooms IS SELECT id, max_occupancy FROM Room;
    v_id      Room.id%TYPE;
    v_max     Room.max_occupancy%TYPE;
BEGIN
    OPEN c_rooms;
    LOOP
        FETCH c_rooms INTO v_id, v_max;
        EXIT WHEN c_rooms%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('ID: ' || v_id || ' Max: ' || v_max);
    END LOOP;
    CLOSE c_rooms;
END;
```

D. Cursores

D.1 ¿Qué es un cursor implícito? ¿Para qué sirve?

Un cursor implícito es aquel que Oracle crea, abre, recupera y cierra automáticamente cada vez que se ejecuta una sentencia SQL (SELECT INTO, INSERT, UPDATE, DELETE) dentro de un bloque PL/SQL. El programador no lo declara ni lo administra explícitamente.

Sirve para ejecutar sentencias SQL que trabajan sobre un conjunto de filas sin requerir una iteración manual, especialmente útil cuando se espera un único resultado (SELECT INTO) o cuando se ejecutan operaciones DML masivas.

Oracle expone sus atributos mediante el cursor especial SQL%:

Atributo	Significado
SQL%FOUND	TRUE si la última sentencia afectó al menos una fila.
SQL%NOTFOUND	TRUE si la última sentencia no afectó ninguna fila.
SQL%ROWCOUNT	Número de filas afectadas por la última sentencia DML.
SQL%ISOPEN	Siempre FALSE para cursores implícitos (Oracle los cierra automáticamente).

```
BEGIN
    UPDATE Room SET max_occupancy = max_occupancy + 1 WHERE id = 301;
    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Filas actualizadas: ' || SQL%ROWCOUNT);
    END IF;
END;
```

D.2 ¿Qué es un cursor explícito? ¿Para qué sirve?

Un cursor explícito es aquel que el programador declara, abre, recorre y cierra manualmente en el bloque PL/SQL. Se usa cuando la consulta puede retornar múltiples filas y se necesita procesarlas una por una.

Su ciclo de vida tiene cuatro pasos: DECLARE → OPEN → FETCH → CLOSE.

Atributos disponibles:

Atributo	Significado
cursor%FOUND	TRUE si el último FETCH recuperó una fila.
cursor%NOTFOUND	TRUE si el último FETCH NO recuperó ninguna fila (fin del resultado).
cursor%ROWCOUNT	Número de filas recuperadas hasta el momento.
cursor%ISOPEN	TRUE si el cursor está abierto actualmente.

También se puede usar el FOR loop implícito sobre cursores explícitos, que automatiza OPEN/FETCH/CLOSE:

```
DECLARE
  CURSOR c_rooms IS
    SELECT r.id, r.max_occupancy, rt.description
    FROM Room r JOIN Room_type rt ON r.room_type_id = rt.id;
  v_row c_rooms%ROWTYPE;
BEGIN
  -- Forma clásica con OPEN/FETCH/CLOSE
  OPEN c_rooms;
  LOOP
    FETCH c_rooms INTO v_row;
    EXIT WHEN c_rooms%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_row.id || ' | ' || v_row.description);
  END LOOP;
  CLOSE c_rooms;
END;
```

D.3 ¿Cuáles son las excepciones propias de uso de estos cursores?

PL/SQL define las siguientes excepciones predefinidas relacionadas con el uso de cursores:

Excepción	Tipo de cursor	Cuándo se produce
NO_DATA_FOUND	Implícito (SELECT INTO)	La consulta no retorna ninguna fila. Código ORA-01403.
TOO_MANY_ROWS	Implícito (SELECT INTO)	La consulta retorna más de una fila cuando se espera exactamente una. Código ORA-01422.
INVALID_CURSOR	Explícito	Se intenta operar (FETCH, CLOSE) sobre un cursor que no ha sido abierto, o ya fue cerrado. Código ORA-01001.
CURSOR_ALREADY_OPEN	Explícito	Se intenta abrir un cursor que ya está abierto con OPEN. Código ORA-06511.

Ejemplo de manejo de excepciones de cursor:

```
DECLARE
    v_max    Room.max_occupancy%TYPE;
BEGIN
    SELECT max_occupancy INTO v_max FROM Room WHERE id = 999;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No existe habitación con ese ID.');
```

```
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('La consulta retornó más de una fila.');
```

```
    WHEN INVALID_CURSOR THEN
        DBMS_OUTPUT.PUT_LINE('Error: cursor no está abierto.');
```

```
END;
```

E. Modularidad

E.1 ¿Cuál es la estructura general de un bloque PL/SQL?

Todo código PL/SQL se organiza en bloques. La estructura general es:

```
[DECLARE
    -- Declaración de variables, constantes, cursores, tipos, etc.
    variable    tipo;
    constante   CONSTANT tipo := valor;
]
BEGIN
    -- Cuerpo: sentencias SQL y de control (obligatorio)
    sentencias_ejecutables;
[EXCEPTION
    -- Manejadores de errores (opcional)
    WHEN nombre_excepcion THEN
        instrucciones_de_manejo;
]
END;
/
```

Sección	Obligatoria	Propósito
DECLARE	No	Declarar variables, constantes, cursores y tipos locales.
BEGIN ... END	Sí	Contiene la lógica ejecutable: SQL, control de flujo, llamadas a subprogramas.
EXCEPTION	No	Captura y maneja errores/excepciones predefinidas o de usuario.

E.2 ¿Para qué sirven las diferentes estructuras modulares?

PL/SQL ofrece cuatro tipos de estructuras modulares principales:

1. Bloque Anónimo (Anonymous Block)

Es un bloque PL/SQL sin nombre que se ejecuta directamente y no se almacena en la base de datos. No puede ser llamado por otros programas.

- Uso: pruebas rápidas, scripts de migración puntual, inicialización de datos.
- Característica: se ejecuta en el momento, desaparece al finalizar.

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hola desde bloque anónimo');
END;
/
```

2. Procedimiento (PROCEDURE)

Es un subprograma nombrado que se compila y almacena permanentemente en la base de datos. Puede recibir parámetros IN, OUT o IN OUT. No retorna valor directamente (usa parámetros OUT para eso).

- Uso: encapsular lógica de negocio reutilizable (Ad, Mo, El del caso de uso).
- Ventaja: puede ser invocado desde otros bloques, aplicaciones o herramientas.

```
CREATE OR REPLACE PROCEDURE add_room(
    p_id          IN Room.id%TYPE,
    p_room_type_id IN Room.room_type_id%TYPE,
    p_max_occ     IN Room.max_occupancy%TYPE DEFAULT NULL
) AS
BEGIN
    -- lógica de inserción
    INSERT INTO Room VALUES (p_id, p_room_type_id, p_max_occ);
END add_room;
/
```

3. Función (FUNCTION)

Similar al procedimiento, pero siempre retorna un valor mediante la cláusula RETURN. Se puede usar directamente en expresiones SQL o PL/SQL.

- Uso: cálculos reutilizables que producen un resultado (ej: calcular precio máximo para un tipo de habitación).
- Restricción: no debe tener efectos secundarios si se usa en SQL (no DML dentro de la función).

```
CREATE OR REPLACE FUNCTION get_max_occupancy(  
    p_type_id IN Room_type.id%TYPE  
) RETURN NUMBER AS  
    v_max NUMBER;  
BEGIN  
    SELECT MAX(occupancy) INTO v_max FROM Rate WHERE room_type_id =  
    p_type_id;  
    RETURN v_max;  
END get_max_occupancy;  
/
```

4. Disparador (TRIGGER)

Es un bloque PL/SQL que se ejecuta automáticamente cuando ocurre un evento DML (INSERT, UPDATE, DELETE) o DDL sobre una tabla o vista. No se llama explícitamente; Oracle lo dispara en respuesta al evento.

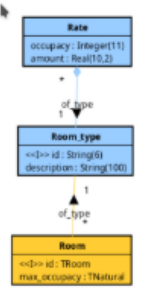
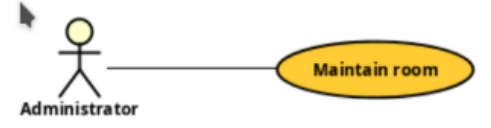
- Uso: automatizar reglas de negocio, auditoría, validaciones complejas que no se pueden expresar con restricciones declarativas.
- Tipos: BEFORE/AFTER + INSERT/UPDATE/DELETE + FOR EACH ROW (nivel fila) o sin él (nivel sentencia).
- En el caso de uso: TR_ROOM_BI (BEFORE INSERT) y TR_STAFF_BU (BEFORE UPDATE).

```
CREATE OR REPLACE TRIGGER TR_ROOM_BI
BEFORE INSERT ON Room
FOR EACH ROW
DECLARE
    v_max_occ NUMBER;
BEGIN
    -- Si no se indica ocupación, calcular la máxima del tipo
    IF :NEW.max_occupancy IS NULL THEN
        SELECT MAX(occupancy) INTO v_max_occ
        FROM Rate WHERE room_type_id = :NEW.room_type_id;
        IF v_max_occ IS NULL THEN
            RAISE_APPLICATION_ERROR(-20001, 'El tipo de habitacion no
            tiene precios asignados.');
```

Estructura	¿Tiene nombre?	¿Retorna valor?	¿Se llama explícitamente?
Bloque anónimo	No	No	Sí (directo, una vez)
Procedimiento	Sí (almacenado)	No (usa OUT)	Sí (CALL / EXEC)
Función	Sí (almacenada)	Sí (RETURN)	Sí (en expresión)
Disparador	Sí (almacenado)	No	No (automático por evento DML)

PRACTICANDO.

En este auto-estudio vamos a implementar un caso de uso simplificado de la base de datos **Guest House**

 <pre>classDiagram class Rate { occupancy: Integer(11) amount: Real(10,2) } class RoomType { <<id>>id: String(8) description: String(100) } class Room { <<id>>id: TRoom max_occupancy: TNatural } Rate "1" -- "*" RoomType : of_type RoomType "1" -- "*" Room : of_type</pre>	TRoom Integer(3) Es un número de 101 a 999, descartando los múltiplos de 100. TNatural Integer(11) Debe ser mayor o igual a 1
 <pre>sequenceDiagram actor Administrator usecase MaintainRoom as Maintain room</pre>	Ad Si no se indica la máxima ocupación, se debe asignar la ocupación mayor considerada en los precios de ese tipo de habitación. No se puede crear si el tipo de habitación no tiene precios asignados. Los cuartos del primer piso no pueden tener más de 2 ocupantes. Mo El único dato que se puede modificar la máxima ocupación. Únicamente puede aumentar. El Se puede eliminar sin restricciones Co Consulta todos los datos básicos de la habitación: identificador, tipo de habitación y máxima ocupación. NOTA: Si se borra un tipo de habitación se deben eliminar todas las habitaciones.

Consultar la guía de diseño que se presenta al final del auto-estudio.

Consultar los **OBJ** en la sección Proyecto del Curso correspondiente a: **Restricciones Declarativas y Restricciones Declarativas, Procedimentales y Automatización.** en la sección Proyecto del Curso correspondiente a: **Restricciones Declarativas y Restricciones Declarativas, Procedimentales y Automatización.**

A. Adicionando restricciones declarativas.

Para proteger la base de datos vamos a implementar las restricciones que se pueden garantizar usando restricciones declarativas.

1. Crear la nueva tabla.

(Tablas, XTablas)

2. Definir las restricciones declarativas

(Atributos, Tuplas)

3. Validar con casos significativos la protección de la base de datos, adicione comentarios que expliquen la condición que están validando.

(AtributosOK, AtributosNoOK, TuplasOK, TuplasNoOK)